Multiplayer Game Platform — Initial Project Plan

Developed by Bogo Solutions

Overview

Three one-week sprints covering the full implementation of the platform as designed in the architecture document. Each sprint is organized by sub-team: **Platform Core**, **Client/UI**, **Rules & Validation**, and **Quality, Testing & Integration**.

Sprint 1 builds the foundation. Sprint 2 implements the core platform features. Sprint 3 integrates everything, closes out stats and notifications, and prepares the demo.

Table of Contents

1. Sprint 1 — Foundation & Core Abstractions
2. Sprint 2 — Core Platform Features
3. Sprint 3 — Integration, Notifications & Demo
4. Dependency Summary
5. Product Backlog

Sprint 1 — Foundation & Core Abstractions (Week 1)

Goal: Establish the project skeleton, all shared interfaces and data models, authentication flows, and stub boundaries so every sub-team can work in parallel from Week 2 onward.

Platform Core

Task	Backlog ID
Project structure setup (`ui` , `platform` , `game` , `model` packages)	SESS-01
Define `Game` interface	SESS-01
Define `RuleValidator` interface	SESS-01
Define `PlayerAction` abstraction	SESS-01
Define `GameStateUpdate` event class	SESS-03
Define `GameStatus` enum and `GameState` class	SESS-02

Task	Backlog ID
Implement `EventBus`	SESS-03
Stub: `CredentialsDatabase`	AUTH-01
Stub: `Database`	PROF-01
Stub: `GameServer` and `Networking`	SESS-02

Client/UI

Task	Backlog ID
`WelcomePage`	UI-01
`AuthenticationScreen`	UI-01
`LoginPage`	UI-01
`State` enum	UI-01

Rules & Validation

Task	Backlog ID
`Player` domain model	PROF-01
`GeneralStats` class	PROF-01
`Rank` enum	PROF-02
`PlayerStats` class	PROF-01
`Settings` class	AUTH-01

Quality, Testing & Integration

Task	Backlog ID
Issue Board Setup	QA-01
README	QA-01
Branching convention	QA-01
Unit tests: `Player`	QA-02

Task	Backlog ID
Unit tests: `GeneralStats`	QA-02
Unit tests: `CredentialsDatabase` stub	QA-02

Sprint 2 — Core Platform Features (Week 2)

Goal: Implement the full matchmaking-to-game-session pipeline, the move handling pipeline, both game implementations, and the primary UI screens. By end of this sprint the lobby → game start → turn progression flow should be runnable end-to-end.

Platform Core

Task	Backlog ID
`SessionManager`	AUTH-02
`PlatformCore` — auth delegation (login & register)	AUTH-01
`PlatformCore` — dispatch & notification routing	SESS-03
`PlatformCore` — queue proxy (`joinQueue`)	MM-01
`PlatformCore` — lobby proxy (`joinLobby`)	MM-02
`PlatformCore` — move proxy (`submitMove`)	SESS-02
`MatchMaking` service	MM-01
`Lobby` service (`createLobby` , `getAvailableLobbies` , `addPlayerToLobby`)	MM-02
`GameSessionManager`	SESS-02
`Leaderboard` service (skeleton)	PROF-02

Client/UI

Task	Backlog ID
`HomePage`	UI-02
`MainGameDashboard`	UI-02
`ProfileDashboard`	UI-04
`PlayGame`	UI-03

Task	Backlog ID
`GameGUI`	UI-03
`ChatboxGUI`	UI-03

Rules & Validation

Task	Backlog ID
`TicTacToeGame` (implements `Game`)	GAME-01
`TicTacToePlayerAction`	GAME-01
`TicTacToeRuleValidator` (implements `RuleValidator`)	GAME-01
`TicTacToeGameState`	GAME-01
`ConnectFourGame` (implements `Game`)	GAME-02
`ConnectFourPlayerAction`	GAME-02
`ConnectFourRuleValidator` (implements `RuleValidator`)	GAME-02
`ConnectFourGameState`	GAME-02

Quality, Testing & Integration

Task	Backlog ID
Merge Sprint 1 branches	QA-03
End-to-end test	QA-03
Update architecture doc	QA-01
Unit tests: `TicTacToeRuleValidator`	QA-02
Unit tests: `ConnectFourRuleValidator`	QA-02
Unit tests: `GameSessionManager`	QA-02
Unit tests: `MatchMaking`	QA-02
Integration test: Login flow	QA-02
Integration test: Register flow	QA-02

Sprint 3 — Integration, Notifications & Demo (Week 3)

Goal: Complete the notification system and stats pipeline, resolve all integration issues, implement the full game-over flow, wire up the leaderboard, and prepare the demo deliverable.

Platform Core

Task	Backlog ID
`Chat` service (`sendMessage` , `filterMessage`)	MM-02
`PlatformCore` — notification routing (complete)	SESS-03
`StatsManager`	PROF-01
`Leaderboard` (complete)	PROF-02
Game-over pipeline (complete)	SESS-02

Client/UI

Task	Backlog ID
`Settings` UI	UI-04
Win/loss/tie result screen	UI-03
Turn alert display	SESS-03
Player statistics view	UI-04
Leaderboard view	PROF-02
UI polish & navigation wiring	UI-02

Rules & Validation

Task	Backlog ID
Win detection — Tic-Tac-Toe	GAME-01
Win detection — Connect Four	GAME-02
Valid actions delivery	SESS-01
Validation failure reason propagation	SESS-01

Quality, Testing & Integration

Task	Backlog ID
Merge Sprint 2 & 3 branches	QA-03
End-to-end demo run	QA-03
Final README	QA-01
Architecture doc update	QA-01
Stub documentation	QA-01
Integration test: full game flow	QA-02
Integration test: turn notification	QA-02
Integration test: game-over stats	QA-02
Unit tests: `StatsManager`	QA-02
Unit tests: win detection	QA-02
Demo script verification	QA-03

Dependency Summary

The table below shows which Sprint 2 and 3 tasks have hard dependencies on Sprint 1 deliverables.

Sprint 2 / 3 Task	Depends On (Sprint 1)
`PlatformCore` auth	`CredentialsDatabase` stub, `SessionManager` skeleton, `Player` model
`GameSessionManager`	`Game` interface, `EventBus`, `GameStateUpdate`
`TicTacToeGame` / `ConnectFourGame`	`Game` interface, `RuleValidator` interface, `PlayerAction` abstraction
`MatchMaking`	`Player` model, `Database` stub
`Lobby`	`Player` model, `GameStatus` enum
`Chat`	`Lobby` service
`StatsManager`	`GeneralStats`, `Rank`, `Database` stub
`PlatformCore` notification routing	`GameStateUpdate`, `EventBus`, `GameStatus` enum

Sprint 2 / 3 Task	Depends On (Sprint 1)
All UI screens	`PlatformCore` API surface, `Player` model, `GameStateUpdate`

Product Backlog

High-level feature areas for the platform. Each sprint task references one of these IDs. Items are ordered by priority within each area.

Priority: `High` — required for a working demo. `Medium` — improves completeness. `Low` — deferred if time is short.

ID	Feature	Priority	Sub-Team
AUTH-01	Authentication & account management (register, login, settings, credentials stub)	High	Platform Core / Rules & Validation
AUTH-02	Session management (create, store, and invalidate session tokens)	High	Platform Core
PROF-01	Player domain model & statistics (Player, GeneralStats, PlayerStats, StatsManager)	High	Rules & Validation / Platform Core
PROF-02	Rankings & leaderboard (Rank enum, Leaderboard service, MMR updates)	Medium	Rules & Validation / Platform Core
MM-01	Ranked matchmaking (queue, eligibility check, auto-match)	High	Platform Core
MM-02	Unranked lobby system (create, browse, join lobby; in-lobby chat)	High	Platform Core
SESS-01	Game abstraction layer (Game interface, RuleValidator, PlayerAction, project structure)	High	Platform Core / Rules & Validation
SESS-02	Game session management (GameSessionManager, GameState, GameServer stub, game-over pipeline)	High	Platform Core
SESS-03	Event & notification system (EventBus, GameStateUpdate, PlatformCore dispatch, turn alerts)	High	Platform Core
GAME-01	Tic-Tac-Toe implementation (Game, RuleValidator, PlayerAction, GameState, win detection)	High	Rules & Validation
GAME-02	Connect Four implementation (Game, RuleValidator, PlayerAction, GameState, win detection)	High	Rules & Validation
UI-01	Authentication UI (WelcomePage, AuthenticationScreen, LoginPage, State enum)	High	Client/UI

ID	Feature	Priority	Sub-Team
UI-02	Dashboard & navigation (HomePage, MainGameDashboard, full screen transition wiring)	High	Client/UI
UI-03	Game UI (PlayGame, GameGUI, ChatboxGUI, result screen)	High	Client/UI
UI-04	Profile & settings UI (ProfileDashboard, Settings UI, player statistics view)	Medium	Client/UI
QA-01	Project infrastructure (GitLab setup, README, branching conventions, documentation)	High	QT&I
QA-02	Testing suite (JUnit infrastructure, unit tests, integration tests)	High	QT&I
QA-03	Integration & release (branch merges, smoke tests, demo run, demo script verification)	High	QT&I

Plan version: aligned with architecture document v1.0.0. Adjust task assignments to sub-team members at the start of each sprint.